Generalized Additive Models and beyond with P-splines

Chapter 1

Dae-Jin Lee
BCAM - Basque Center for Applied Mathematics
XV Summer School MESIO, 2022

Barcelona, July 2022

1. Introduction to smoothing

In this section, we will review:

- Linear models
- Generalized Linear Models

To introduce you to:

- Scatterplot smoothing
- Kernel smoothing and splines

Quick review of Linear models

Remember in the ordinary least squares linear regression model, we have some response/variable y we want to study assumed to be normally distributed with mean μ and variance σ^2.

We also have a set of predictors/covariates or explanatory variables $x_1, x_2, ..., x_p$. A linear model is of the form:

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon} \quad \boldsymbol{\epsilon} \sim N(0, \sigma^2)$$

where $\boldsymbol{X} = [\mathbf{1} : x_1 : x_2 : ... : x_p]$ and $\boldsymbol{\beta} = (\beta_0, \beta_1, ..., \beta_p)'$.

One of the issues with this model is that, in its basic form it can be very limiting in its assumptions about the data generating process for the response variable of interest.

Generalized Linear Models (GLMs)

GLMs incorporate other types of distributions for the response variable y (of the exponential family e.g.: Binomial, Poisson, Gamma, etc . . .), and include a link function $g(\cdot)$ relating the mean μ through the so-called *linear predictor* η. The general form is:

$$g(\mu) = \eta = \boldsymbol{X}\boldsymbol{\beta},$$

where the estimated fitted values $\mathbb{E}[y] = \mu = g^{-1}(\eta)$. For the Poisson distribution, the (canonical) link function $g(\cdot)$, is the natural $\log$, for Binomial is the $logit$ and for Gaussian distribution is the identity.

Scatterplot smoothing

Scatterplot smoothing consists of highlighting the *underlying trend* in the data. The underlying trend would be a function such as:

$$f(x) = \mathbb{E}(y|x)$$

That can also be written as:

$$y_i = f(x_i) + \epsilon_i, \qquad \mathbb{E}(\epsilon_i) = 0,$$

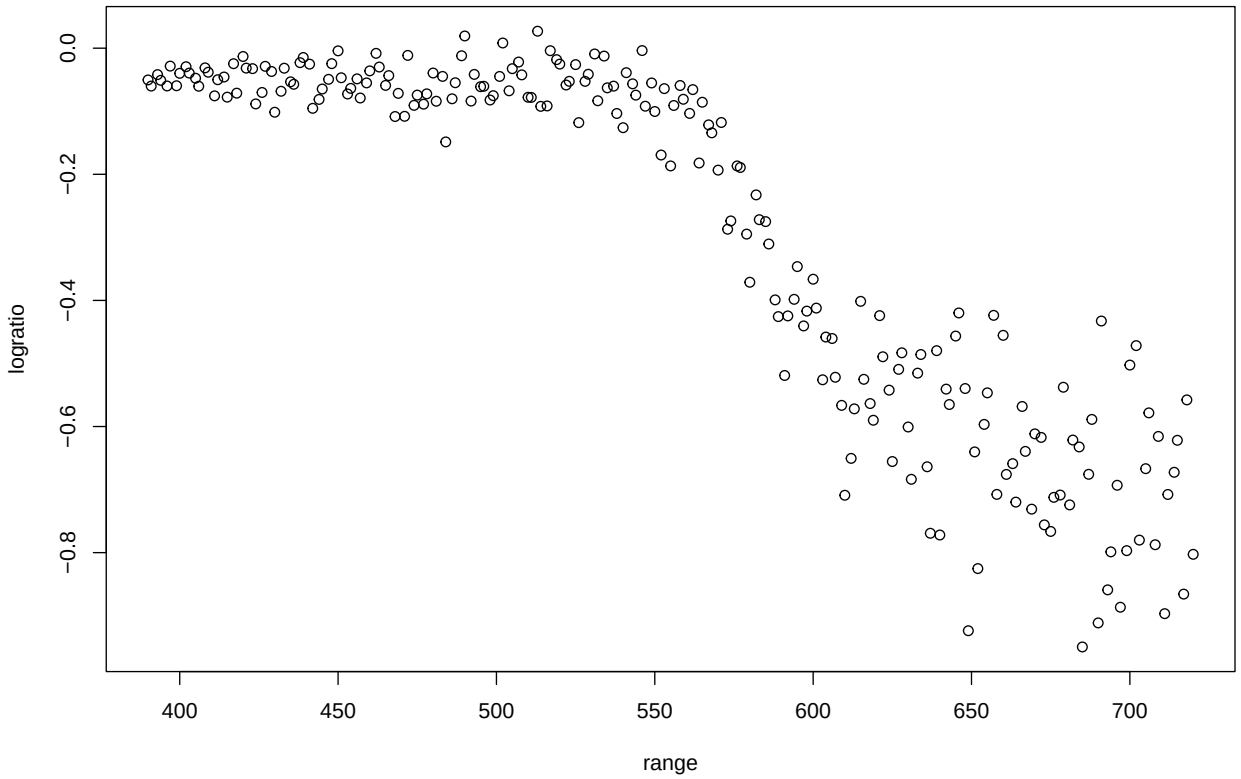
in which case the problem is often referred to as *non-parametric regression*, where $f(\cdot)$ os some unespecified *smooth* function that needs to be estimated from the pair (x_i, y_i)

There are several methods for smoothing a scatterplot, including *splines*, *kernel regression*, *running means*, *loess* or the weighted version *lowess*.

Scatterplot smoothing: LIDAR data

The `lidar` data frame has 221 observations from a light detection and ranging (LIDAR) experiment.

```
library(SemiPar)
data(lidar)
plot(logratio~range,data=lidar)
```



Scatterplot smoothing: LIDAR data (cont.)

Polynomial regression consists of transforming the predictor X, adding p degree polynomials such that

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_p X^p + \epsilon \quad \epsilon \sim N(0, \sigma^2)$$

Usually orthogonal polynomials are used to avoid collinearity.

- The `R` function `poly` evaluates orthogonal polynomials of selected degree p to a set of points `x`. If argument `raw=FALSE` is used, the powers of $x, x^2, ..., x^p$ are used instead.

```
fit.poly <- lm(logratio ~ poly(range,degree = 3), data = lidar)
```

Scatterplot smoothing: LIDAR data (cont.)

```r
summary(fit.poly)
```

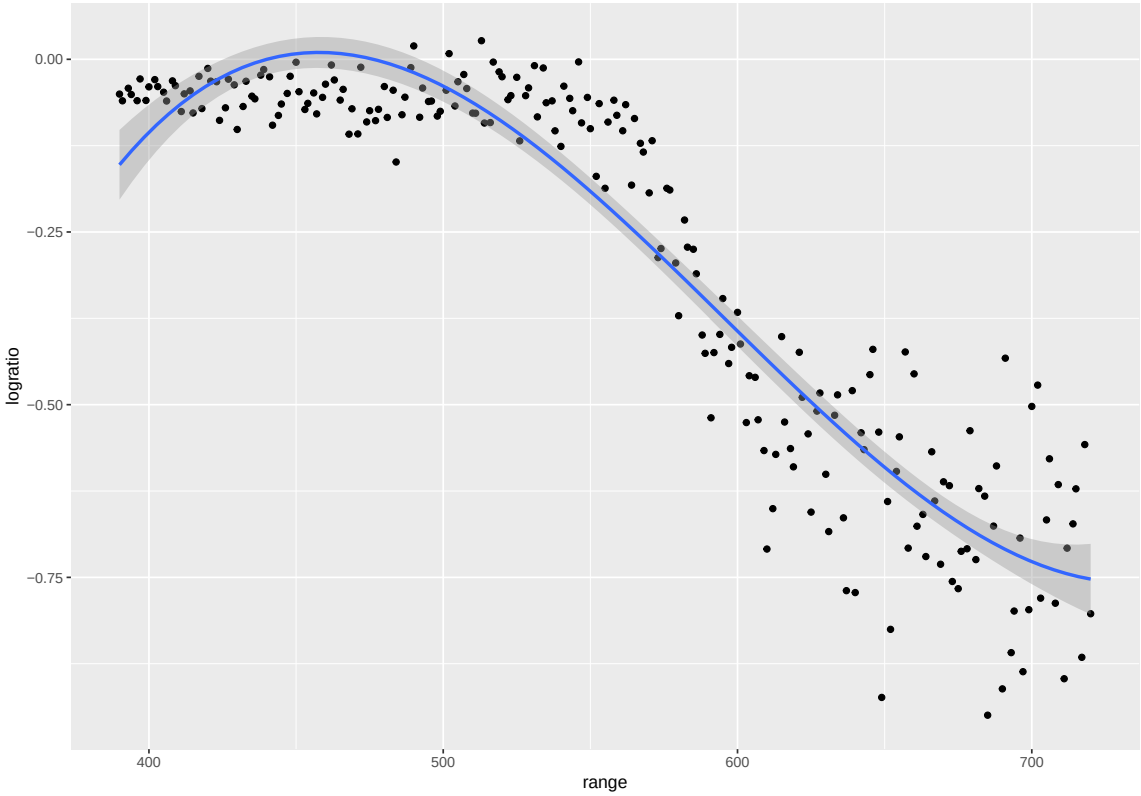
```
##
## Call:
## lm(formula = logratio ~ poly(range, degree = 3), data = lidar)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.33671 -0.05952 -0.00521  0.06489  0.27691
##
## Coefficients:
##                           Estimate Std. Error t value Pr(>|t|)
## (Intercept)              -0.291156   0.006542 -44.502  < 2e-16 ***
## poly(range, degree = 3)1 -3.706758   0.097261 -38.112  < 2e-16 ***
## poly(range, degree = 3)2 -1.091555   0.097261 -11.223  < 2e-16 ***
## poly(range, degree = 3)3  0.754951   0.097261   7.762 3.25e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.09726 on 217 degrees of freedom
## Multiple R-squared:  0.8831, Adjusted R-squared:  0.8814
## F-statistic: 546.2 on 3 and 217 DF,  p-value: < 2.2e-16
```

Using ggplot2 and geom_smooth

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.1.2
```

```
ggplot(lidar, aes(range, logratio)) +
  geom_point() +
  geom_smooth(method = "lm", formula = y ~ poly(x, degree = 3),se = TRUE)
```



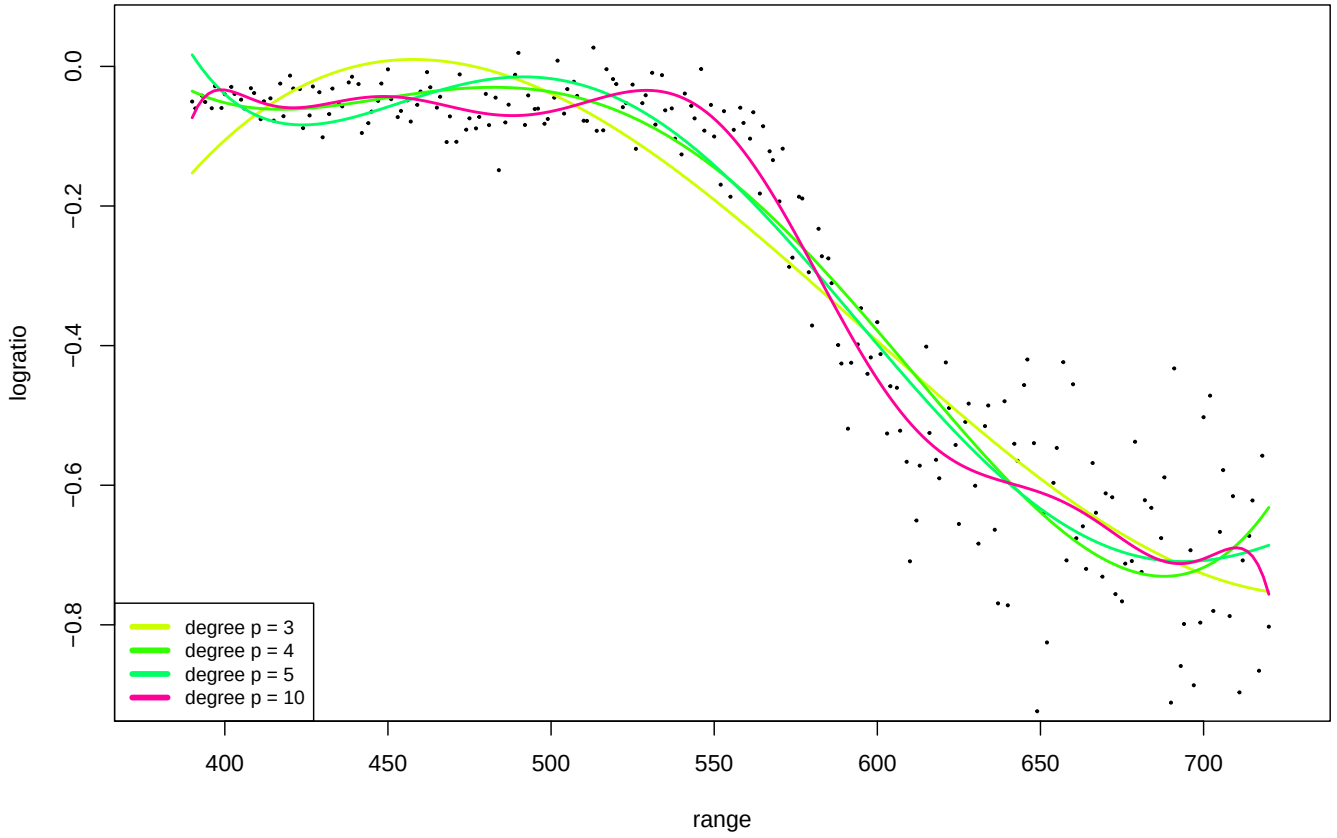


Figure 1: Scatterplot smoothing of LIDAR data with polynomial regression of different degrees

Local regression or local polynomial regression

A smoothing technique that attempts to fit a smooth curve using a weighted local regression technique.

The fit is local because the adjustment at a point x_i uses only observations in an environment around x_i (called a window or span), giving more weight to observations that are closer to x_i and less to those that are further away.

That is, it assumes that nearby values will be more similar to each other with respect to the rest of the values. similar to each other with respect to the rest of the values.

For each point x_i, the procedure would be as follows:

- *Choose the size of the window:* for example, if it is 50% it means that the window contains 50% of the total data with x_i in the centre.
- *Assign weights:* each of the observations in the window is assigned a weight that is greater the closer the observation is to x_i.
- *Weighted least squares fitting:* to obtain the predicted value $\hat{y}_i$ at point x_i we use the points in the local neighborhood of x_i using a weighted local regression model at x_i.

Let us fit the LIDAR data with loess

- The R function `loess.smooth` evaluates a loess curve

```
fit.loess <- with(lidar,loess(logratio~range, span = 1/2, degree = 2))
pred <- predict(fit.loess,se=TRUE) # Obtain standard errors with se=TRUE
pred.se <-pred$se # obtain se
plot(logratio~range,data=lidar, cex = .3,pch=15)
lines(fit.loess$x,pred$fit,col=2,lwd=2)
lines(fit.loess$x,pred$fit+1.96*pred.se,col=1,lwd=1,lty=2)
lines(fit.loess$x,pred$fit-1.96*pred.se,col=1,lwd=1,lty=2)
```

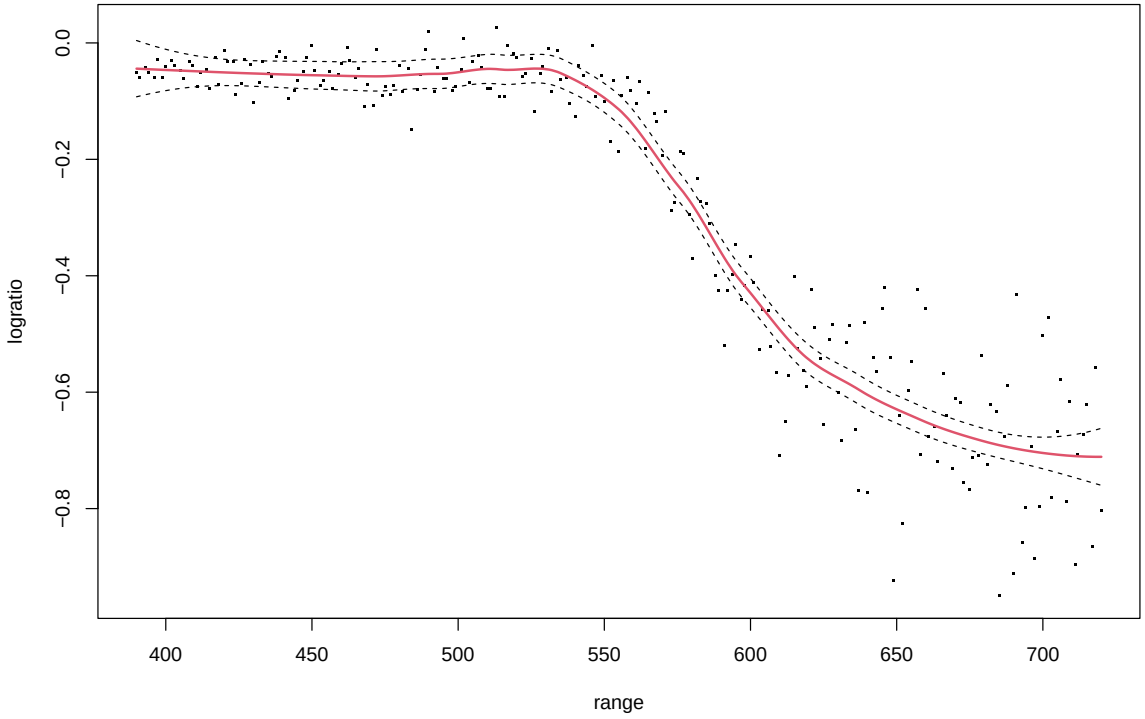
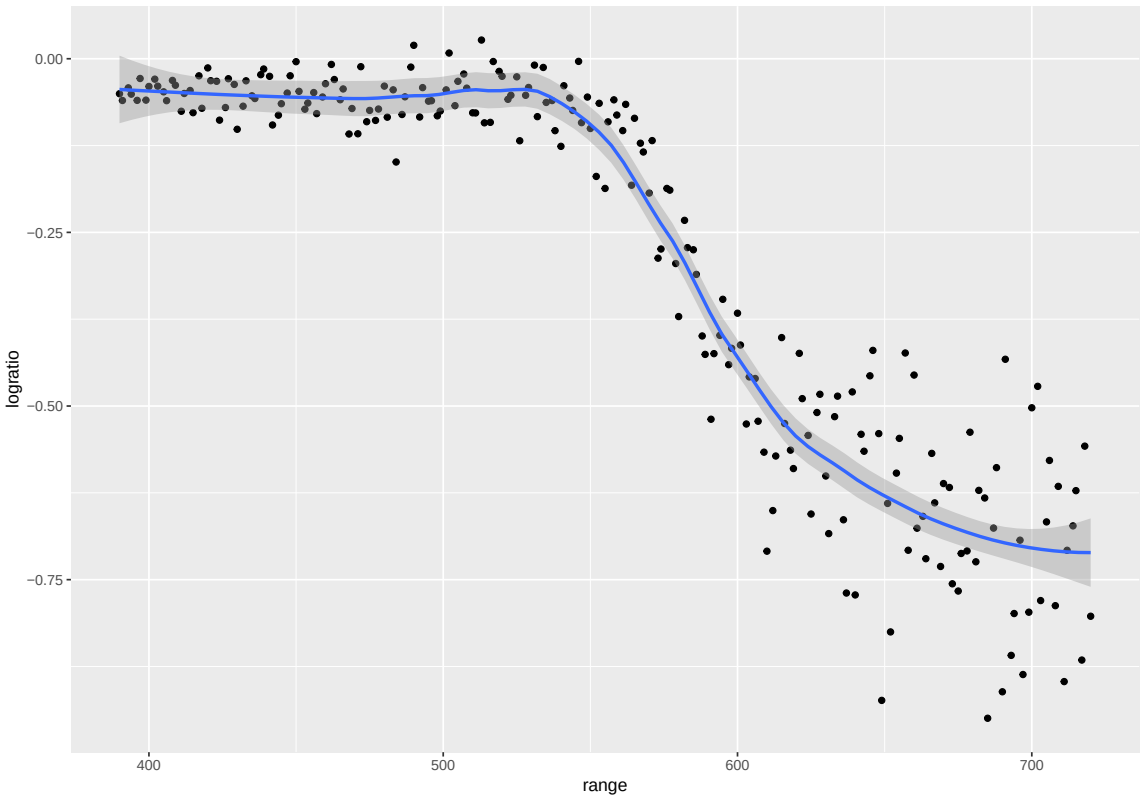


Figure 2: LOESS smoothing with a span of 0.50

Using ggplot2 and geom_smooth

```
library(ggplot2)
ggplot(lidar, aes(range, logratio)) +
  geom_point() +
  geom_smooth(span = 0.50,se = TRUE)
```

```
## `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```



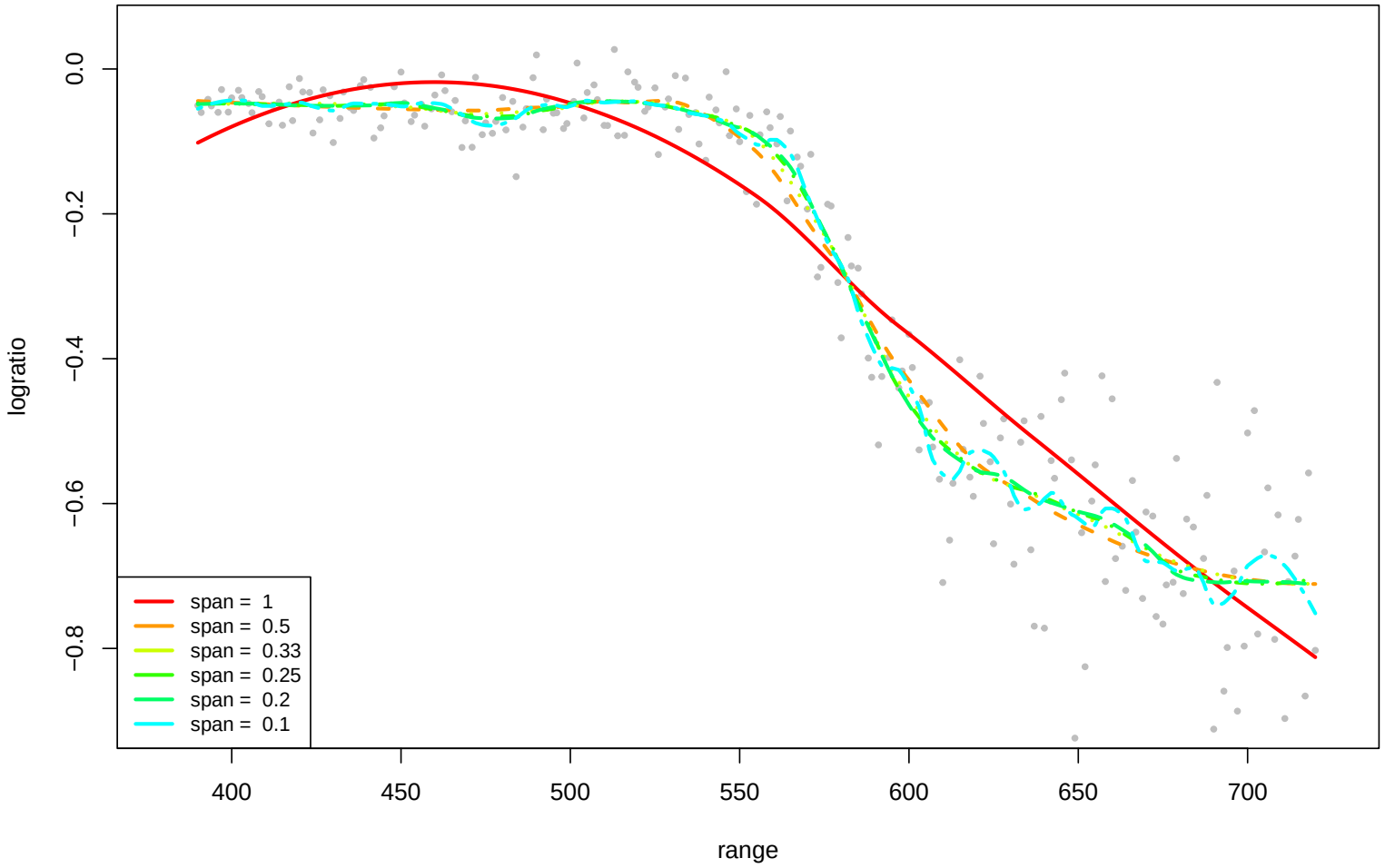


Figure 3: LOESS smoothing with different span

Kernel smoothing

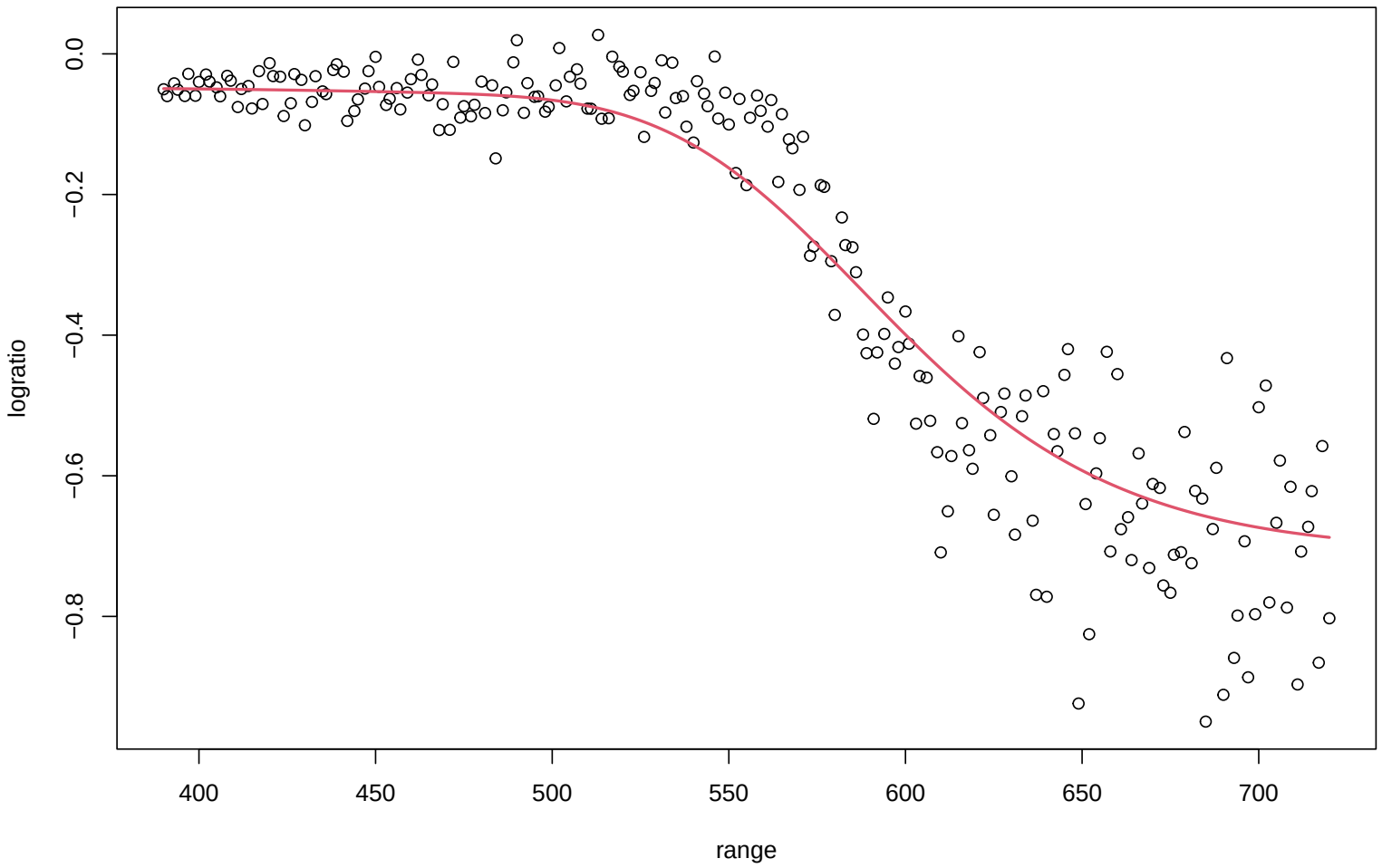
A *kernel smoother* is of the form

$$\hat{y}_i = \frac{\sum_{j=1}^{n} y_i K\left(\frac{x_i - x_j}{b}\right)}{\sum_{j=1}^{n} K\left(\frac{x_i - x_j}{b}\right)},$$

where b is a *bandwidth* parameter, and K a kernel function, as in density estimation. There different choices for the kernel K (e.g.: Gaussian), the critical choice parameter is the bandwidth *b*.

In `R` the function `ksmooth` in the `library(stats)` can be used for kernel smoothing (other options are `locpoly` in `library(KernSmooth)`)

```
fit.kernel <- with(lidar,ksmooth(range,logratio,kernel = "normal", bandwidt
plot(logratio~range,data=lidar)
lines(fit.kernel,col=2,lwd=2)
```



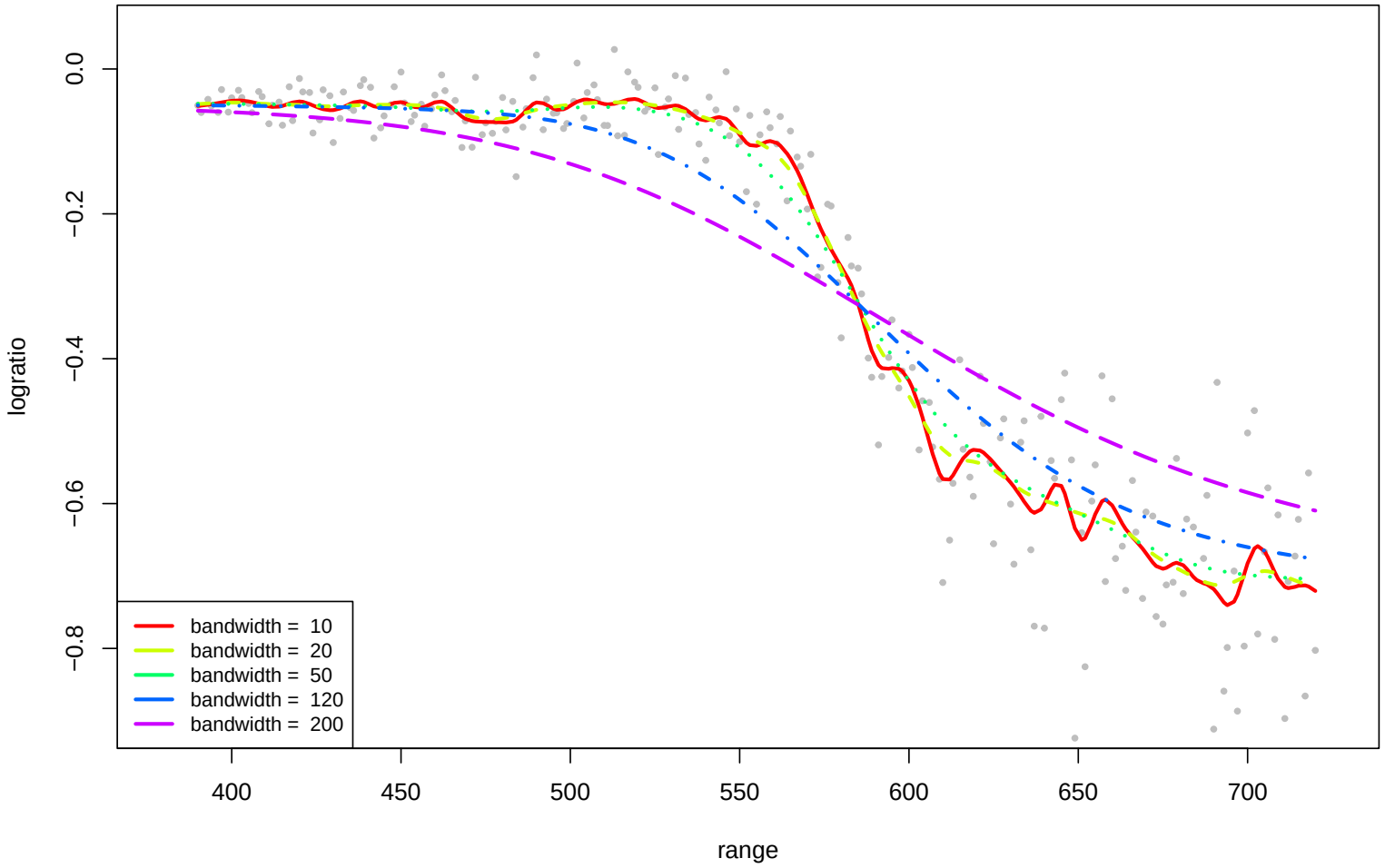


Figure 4: Kernel smoothing with different bandwiths and Gaussian kernel

Spline regression

Spline regression is a non-parametric regression technique. This regression technique divides the datasets into bins at intervals or points called knots and each bin has its separate fit.

- Divide the real line by and ordered set of points $\{z_i\}$ known as *knots*.
- On the interval $[z_i, z_{i+1}]$ the spline is a cubic polynomial, and its continuous and has continous first and second derivatives, imposing conditions at each knot.

A restricted form of B-splines are *natural splines* (implemented in `R` by function `ns` in `library(splines)`) is linear on $(-\infty, z_1]$ and $[z_n, \infty)$ and this have n parameters. However, `ns` adds an extra knot at each of the maximum and minimum of the data points, and so has $n + 2$ parameters, dropping the requirement for the derivative to be continuous at z_1 and z_n.

The arguments of `ns` are

- `df` (degrees of freedom) and `knots`.
- One can supply `df` rather than `knots`, then `df-1-intercept` inner knots at suitable chosen quantiles of x.
- Additionaly an `intercept` can be added to the model (default is `FALSE`)

```
library(splines)
fit.ns <- lm(logratio~ns(range, df = 5),data=lidar)
```

```
##
## Call:
## lm(formula = logratio ~ ns(range, df = 5), data = lidar)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.288489 -0.038180  0.000965  0.045459  0.267075
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)        -0.01923    0.02469  -0.779  0.43694
## ns(range, df = 5)1  0.09249    0.03135   2.951  0.00352 **
## ns(range, df = 5)2 -0.32845    0.03995  -8.221  1.9e-14 ***
## ns(range, df = 5)3 -0.66528    0.03104 -21.432  < 2e-16 ***
## ns(range, df = 5)4 -0.76952    0.06260 -12.293  < 2e-16 ***
## ns(range, df = 5)5 -0.62306    0.02747 -22.684  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.08354 on 215 degrees of freedom
## Multiple R-squared:  0.9145, Adjusted R-squared:  0.9125
## F-statistic:   460 on 5 and 215 DF,  p-value: < 2.2e-16
```

```r
pred <- predict(fit.ns,interval='confidence',se.fit=TRUE)
plot(logratio~range,data=lidar,col='grey',pch=19,cex=.5)
lines(lidar$range,pred$fit[,1],col=2,lwd=2)
lines(lidar$range,pred$fit[,2],col=1,lwd=1,lty=2)
lines(lidar$range,pred$fit[,3],col=1,lwd=1,lty=2)
```

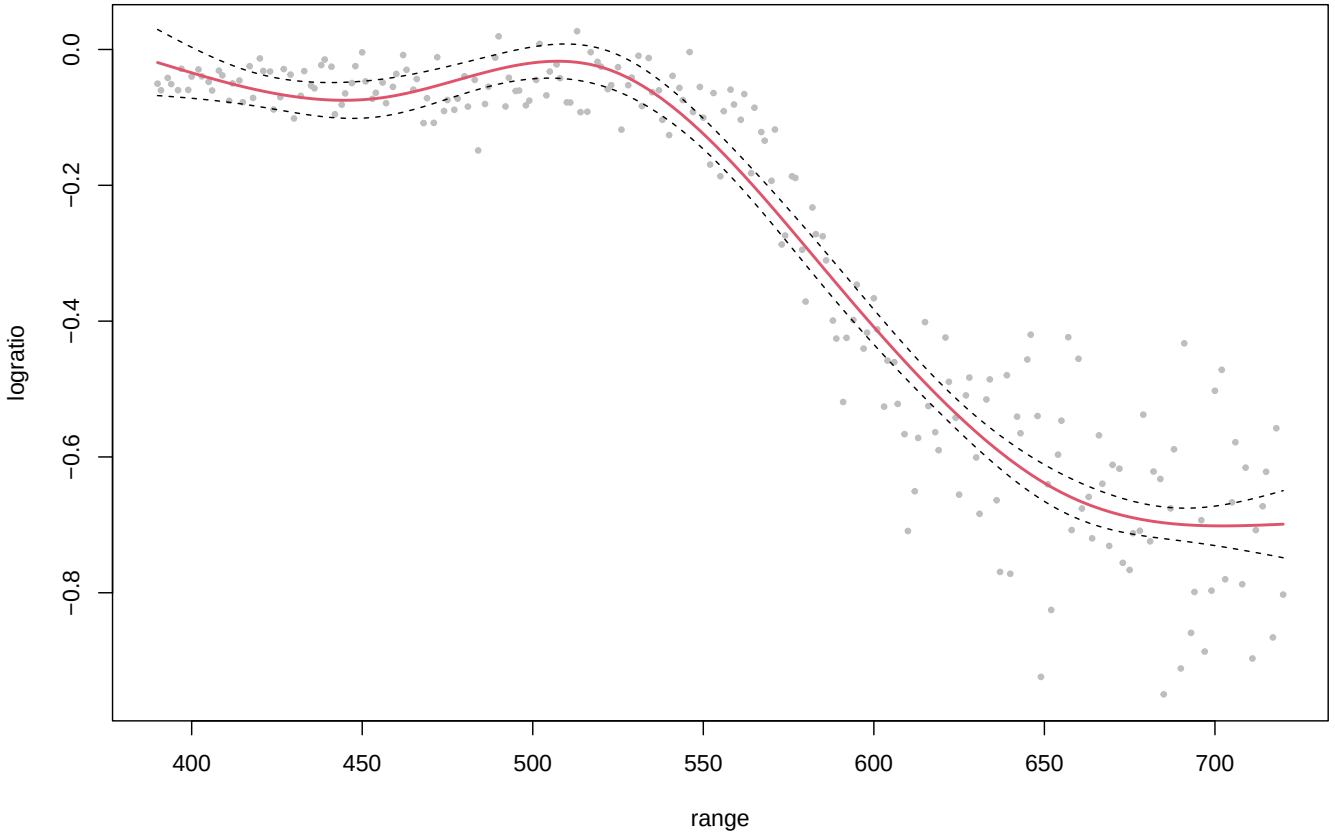


Figure 5: Spline regression fit with ‘df=5‘

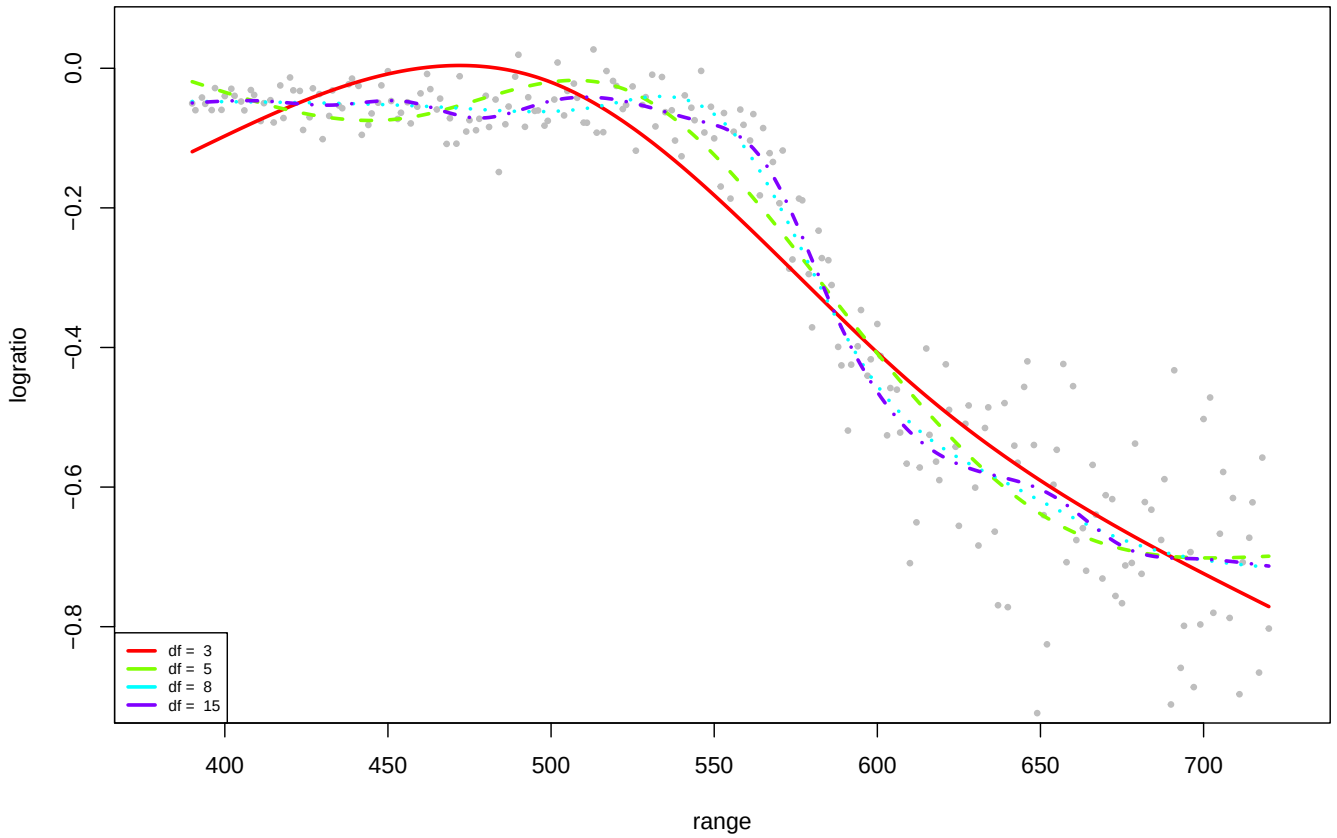
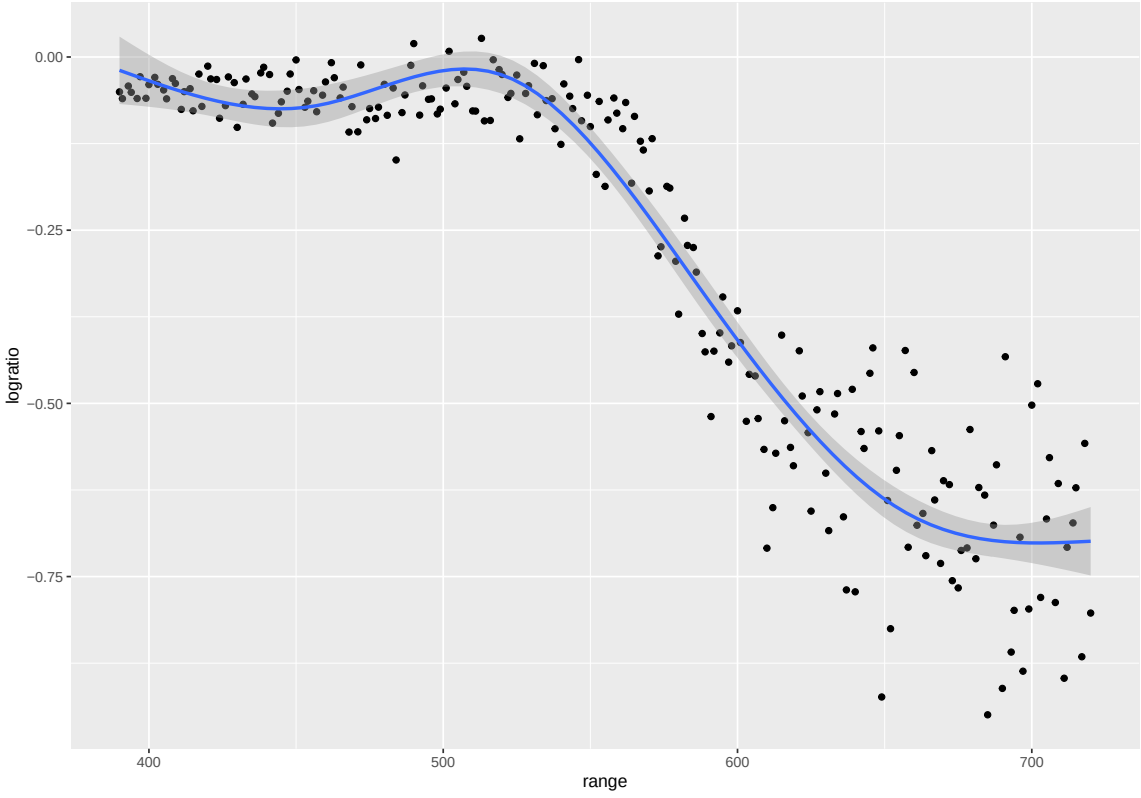


Figure 6: Spline regression with different degrees of freedom

Using ggplot2 and geom_smooth

```
library(ggplot2)
ggplot(lidar, aes(range, logratio)) +
  geom_point() +
  geom_smooth(method = "lm", formula = y ~ ns(x, df = 5),se = TRUE)
```



Smoothing splines

A *smoothing spline* minimizes a compromise between the fit and the degree of smoothness of the form of a residual sum of squares:

$$RSS(f, \lambda) = \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int_{x_1}^{x_n} (f''(x))^2 dx$$

over all (measurably twice-differentiable) functions.

It is a cubic spline with knots at the x_i, but does not interpolate the data points for $\lambda > 0$ (*smoothing parameter*) and the degree of the fit is controlled by λ.

The **smoothing parameter** $0 < \lambda < \infty$

The function `smooth.spline` fits a cubic splines to the data

```
# with fixed df
fit.sp1<-with(lidar,smooth.spline(range,logratio,df=5))

# fixed smoothing parameter (spar)
fit.sp2<-with(lidar,smooth.spline(range,logratio,spar=1))
```

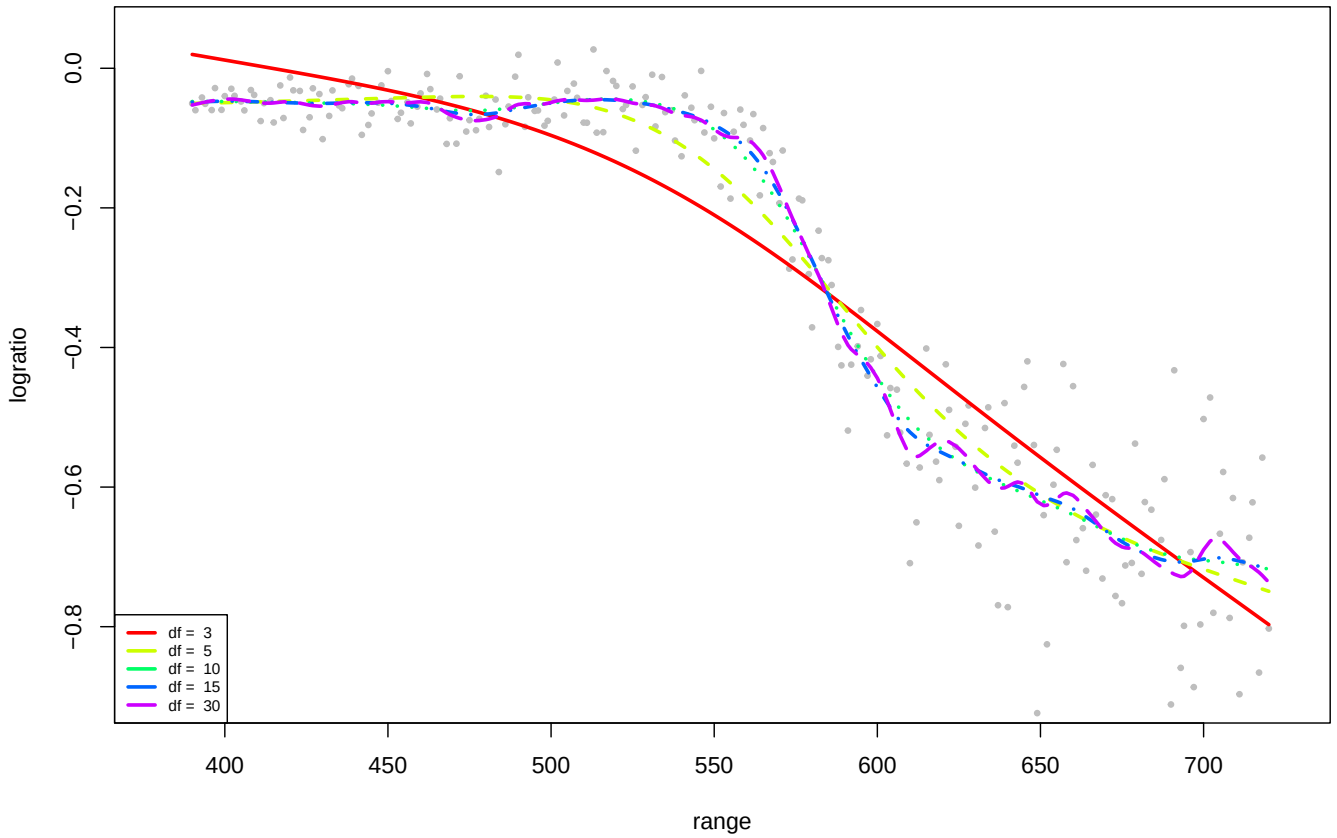


Figure 7: Smoothing splines fit with different degrees of freedom

The `smooth.spline` function chooses automatically the degrees of freedom by cross-validation if the `df` argument is not specified.

The degrees of freedom are the trace of the so-called smoother or hat-matrix H, as fitting a smoothing splines is a linear operation, there is an $n \times n$ matrix H such that $\hat{y} = Hy$.

Until now, we haven't discussed the choice of the optimal smoothing ...

Best smoothing was chosen via selection of the degree of the polynomial or the number of natural splines ...

Choice of the smoothing parameter

Cross-validation

The cross-validation is a general procedure that can be applied to estimate smoothing parameters in a wide variety of problems. Let $\boldsymbol{y}^{-i}$ be the $n-1$ vector with the ith observation, y_i, removed from the original response vector $\boldsymbol{y}$.

Let $\widehat{f}_\lambda^{-i}$ be the estimate based on $n-1$ observations $\boldsymbol{y}^{-i}$. The ordinary cross-validation (CV) estimate of the prediction error is

$$\mathrm{CV}(\lambda) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{f}_\lambda^{-i}(t_i))^2 .$$

A cross-validation estimate of λ is the minimizer of CV. The CV reduces to

$$\mathrm{CV}(\lambda) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \widehat{f}_\lambda(x_i)}{1-h_{ii}}\right)^2 .$$

Thus one only needs to fit the model once with the full data and compute the diagonal elements of the H_λ hat-matrix.

Choice of the smoothing parameter (Cont.)

Generalized Cross-validation

Replacing h_{ii} in CV by the average of all diagonal elements, the following generalized cross-validation (GCV) criterion is proposed

$$\mathrm{GCV}(\lambda) = \frac{\frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{f}_\lambda(t_i))^2}{(1 - trH(\lambda)/n)^2}.$$

Generalized cross-validation is used by default i.e. `cv=FALSE`

```
with(lidar,smooth.spline(range,logratio,cv=FALSE))
```

```
## Call:
## smooth.spline(x = range, y = logratio, cv = FALSE)
##
## Smoothing Parameter  spar= 0.7873307  lambda= 0.0007064533 (11 iterations)
## Equivalent Degrees of Freedom (Df): 9.387695
## Penalized Criterion (RSS): 1.336206
## GCV: 0.006594531
```

For $\lambda = 0$ the smoothing spline will interpolate the data points if the x_i is distinct.

As a linear smoother, for each value of x_i there are a set of basis functions $B_j(x)$ such that:

$$f_\lambda(x) = \sum_{j=1}^{n} \theta_j B_j(x)$$

where θ_j are the coefficients and $B_j(x)$ are the basis functions. Let $\boldsymbol{B}$ be the $n \times n$ matrix defined by

$$B_{ij} = B_j(x_i)$$

and penalty matrix $\boldsymbol{\Omega}$

$$\boldsymbol{\Omega}_{ij} = \int_{x_1}^{x_n} B_i^{''}(x) B_j^{''} dx$$

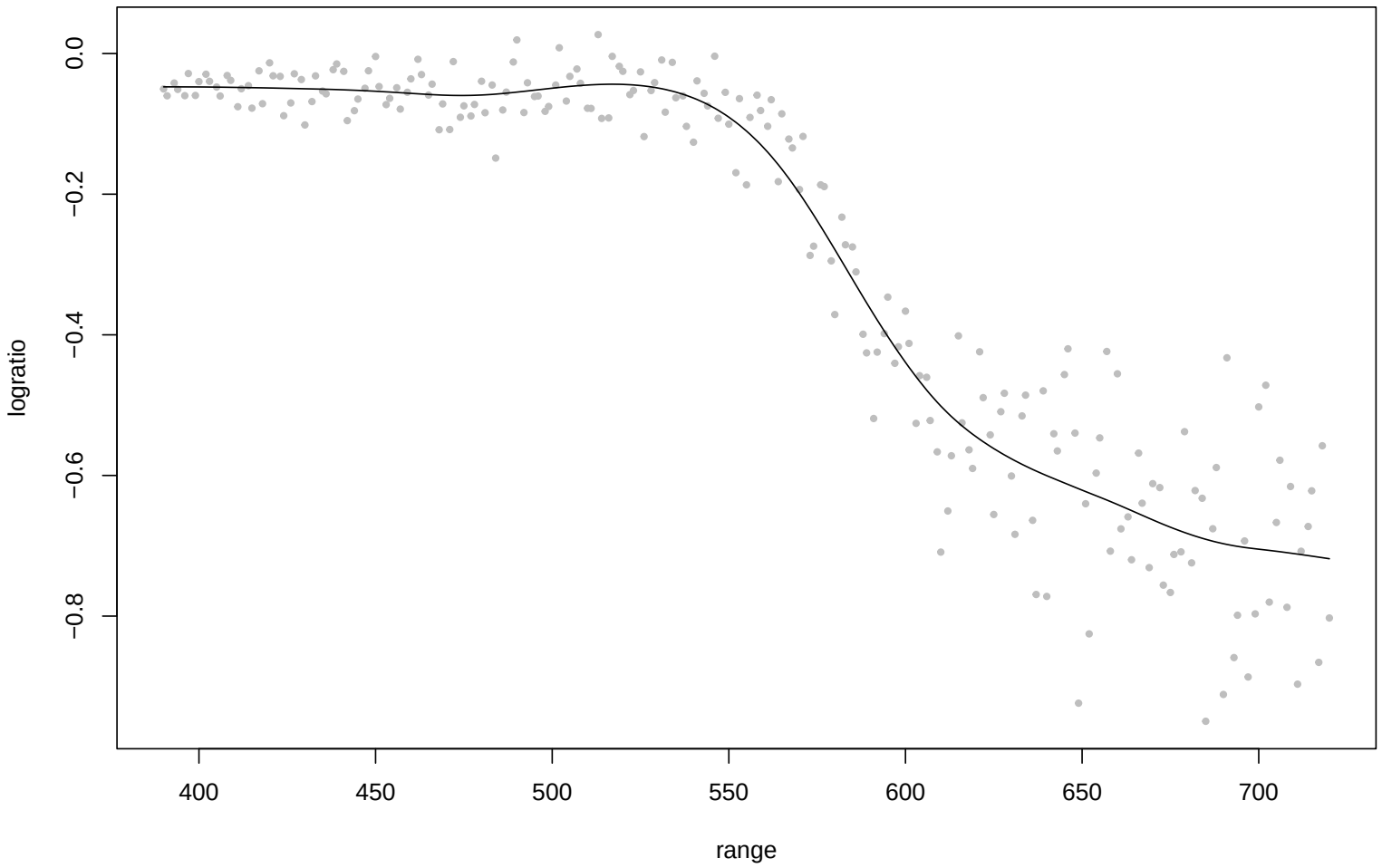
The penalized criterion is the minimization of the penalized residual sum of squares is defined as

$$PRSS(\boldsymbol{\theta}, \lambda) = (\boldsymbol{y} - \boldsymbol{B}\boldsymbol{\theta})'(\boldsymbol{y} - \boldsymbol{B}\boldsymbol{\theta}) + \lambda \boldsymbol{\theta}' \boldsymbol{\Omega} \boldsymbol{\theta}$$

The solution to the minimization of $PRSS$ is

$$\hat{\boldsymbol{\theta}}_\lambda = (\boldsymbol{B}'\boldsymbol{B} + \lambda \boldsymbol{\Omega})^{-1} \boldsymbol{B}' \boldsymbol{y}$$

```r
attach(lidar)
plot(range,logratio,col='grey',pch=19,cex=.55)
lines(smooth.spline(range,logratio,cv=FALSE))
```

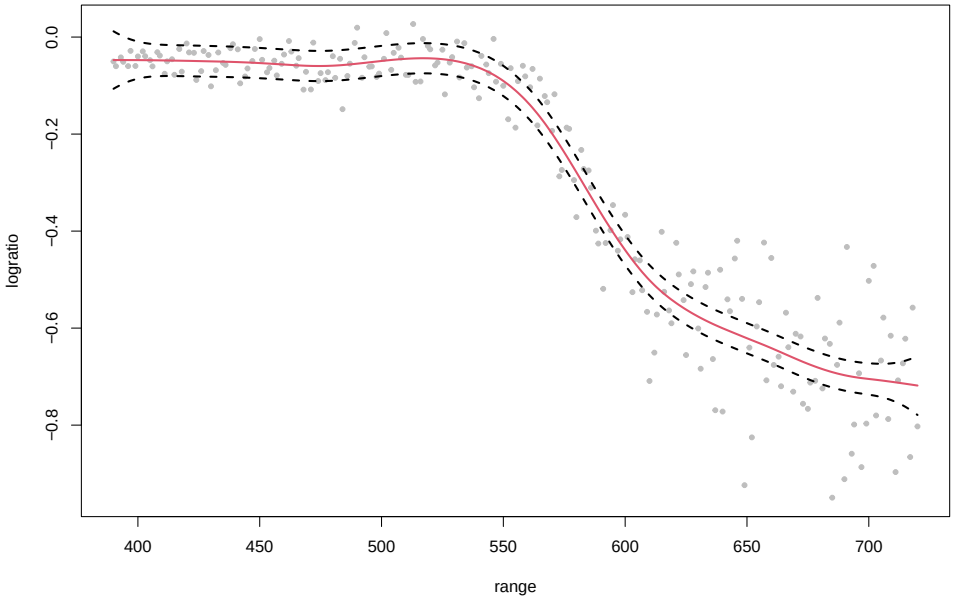


Confidence intervals?

```
attach(lidar)
  fit <- smooth.spline(range, logratio, cv=FALSE)      # smooth.spline fit
  res <- (fit$yin - fit$y)/(1-fit$lev)       # jackknife residuals
sigma <- sqrt(var(res))                      # estimate sd

upper <- fit$y + 1.96*sigma*sqrt(fit$lev)    # upper 95% conf. band
lower <- fit$y - 1.96*sigma*sqrt(fit$lev)    # lower 95% conf. band

plot(range,logratio,col='grey',pch=19,cex=0.65)
matlines(fit$x, cbind(upper, fit$y, lower), type="l", pch=".",col=c(1,2,1),lwd=2,lty=c(2,1,2))
```



Summary of this chapter

- Scatterplot smoothing allow us to find patterns in the data
- There are several smoothing techniques
- The amount of smooothness (selection of the smoothing parameter is a crucial choice)
- Several `R` packages and functions

What's next?

Generalized Additive Models